SIMATS ENGINEERING

ASSESSMENT TOOL 2 (Medium)- Debugging and Optimization

COURSE NAME: Data Structures **COURSE CODE:CSA03 16**

COURSE OUTCOME COVERED

CO2 : Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques. (BTL3, BTL4)

Assessment Tool Weightage: **10%**

1. **Find the bugs, include the missing lines in the following snippet.**

int linearSearch(int arr[], int n, int key) {

for(int i = 0; i <= n; i++) {

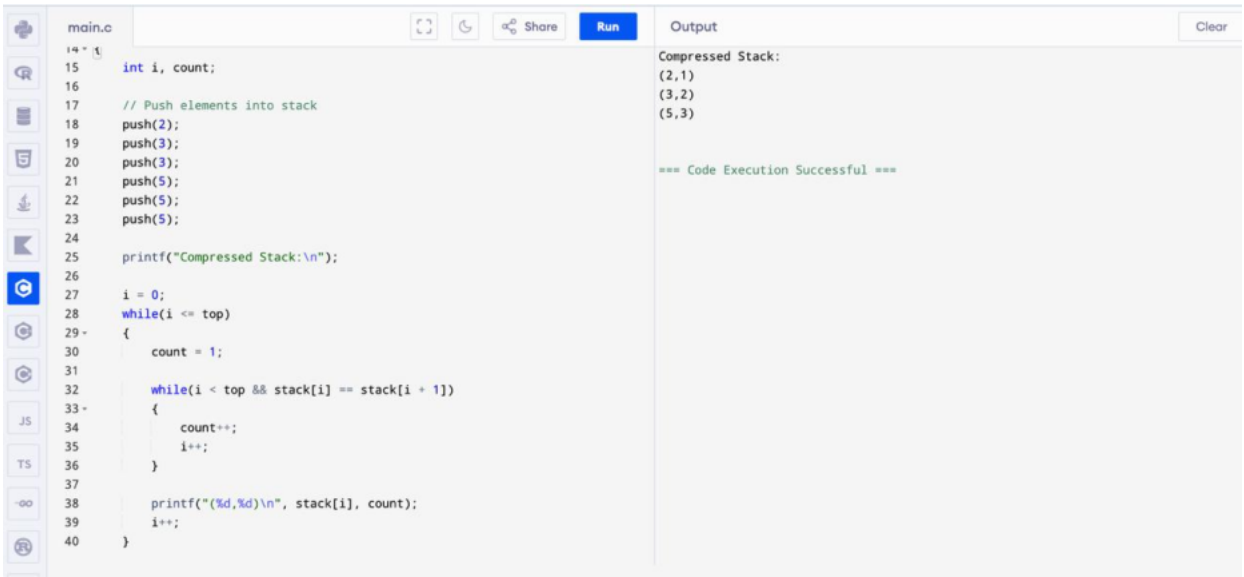
if(arr[i] == key)

return i;

}

return -1;

}

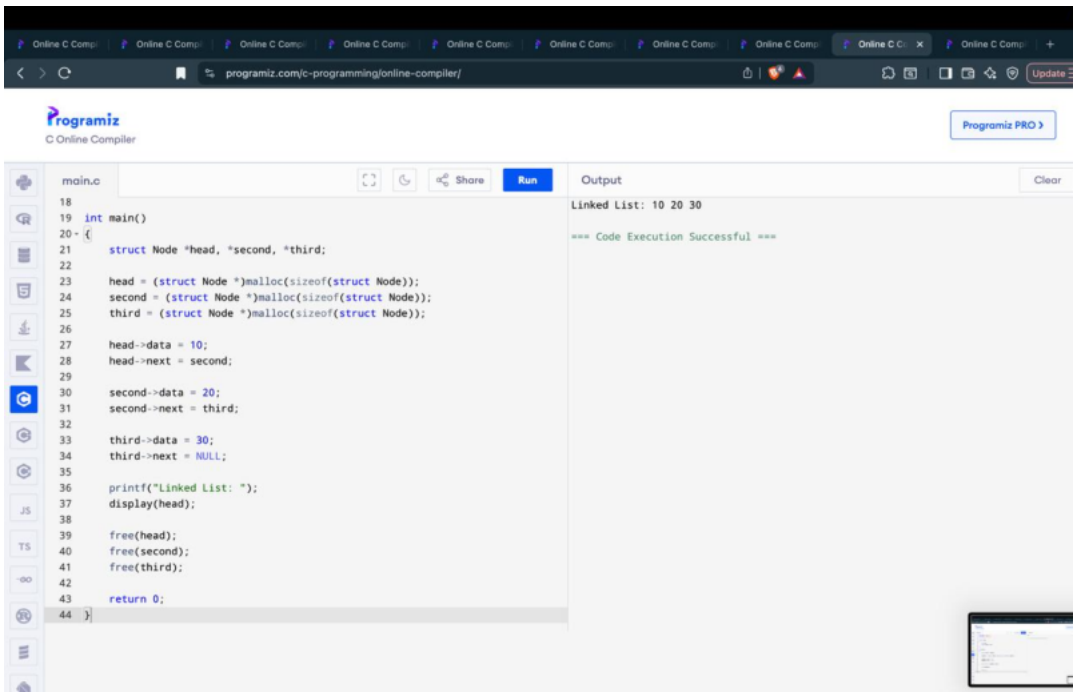


2. **Buggy Code in an array**

```
#include <stdio.h>

int main() {

   int arr[5] = {10,20,30,40,50};

   for(int i=0;i<=5;i++)

      printf("%d ",arr[i]);

   return 0;

}
```

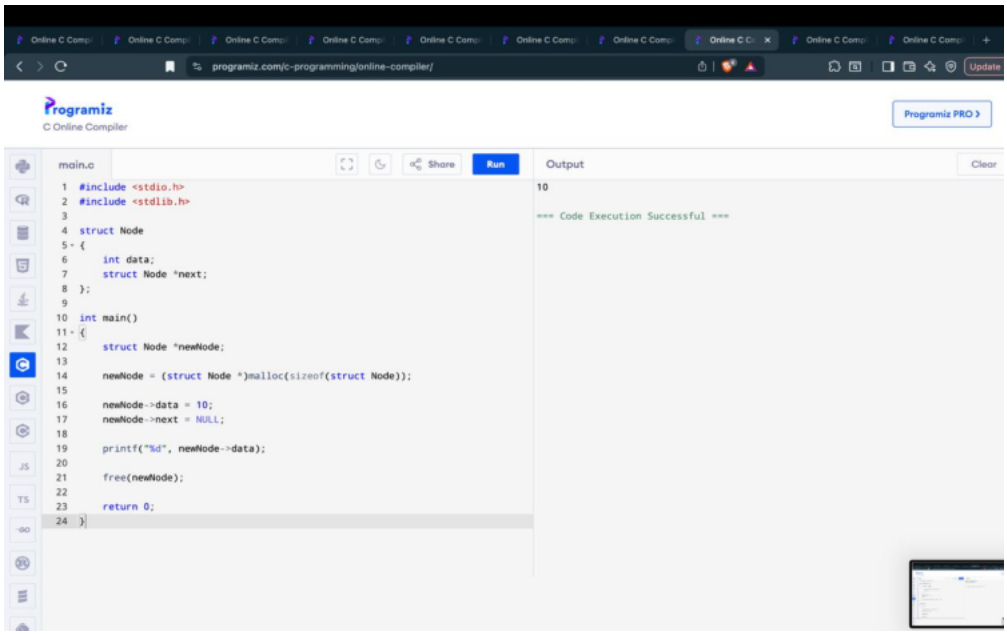
Expected output:

10 20 30 40 50



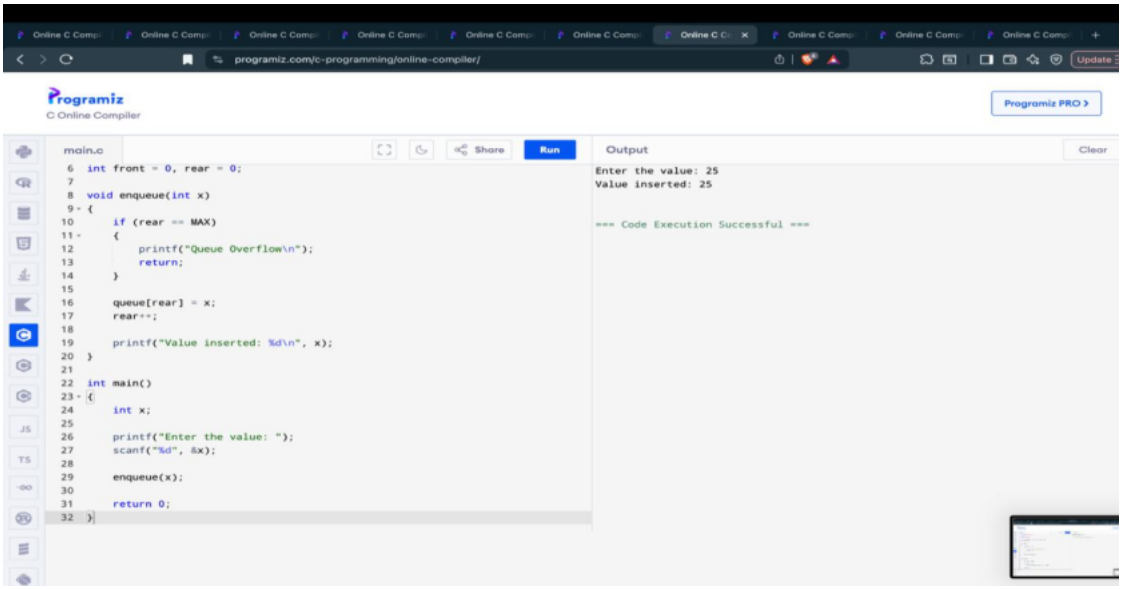
3. Identify an errors in the following snippet

```
int binarySearch(int arr[], int n, int key)
{
 int low = 0, high = n - 1;
while(low < high)
 { int mid = (low + high) / 2;
 if(arr[mid] == key) return mid;
else if(arr[mid] < key)
low = mid;
else high = mid;
 }
return -1;
}
```



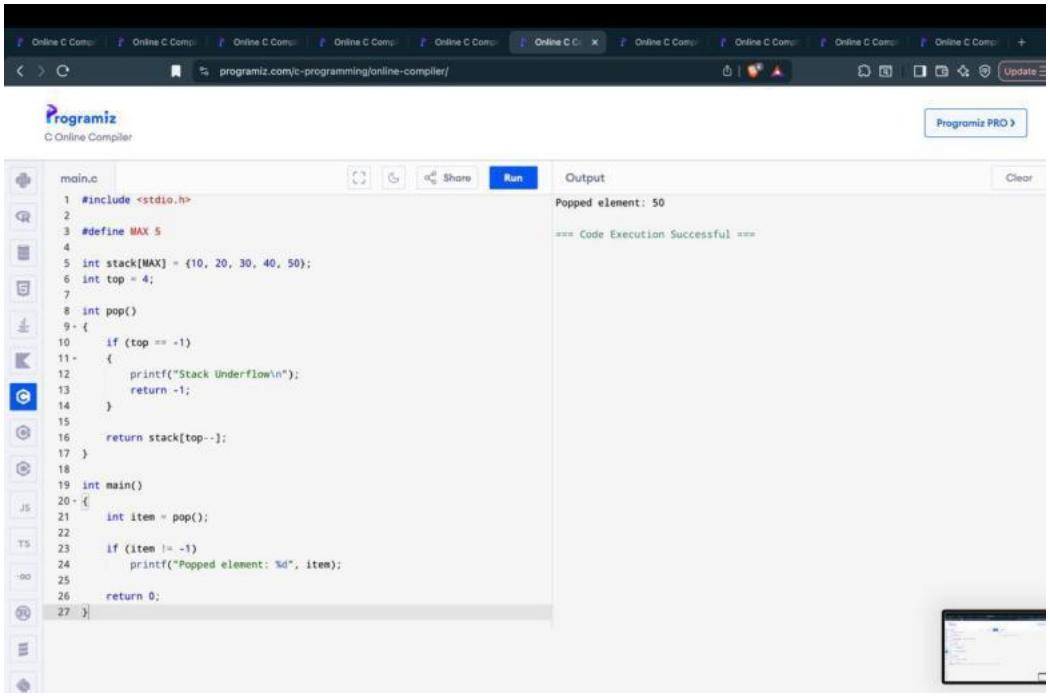
4. **Buggy Code in Stack Push Operation**

```
#include <stdio.h>

#define MAX 5

int stack[MAX];
int top = -1;

void push(int data) {
   if (top == MAX - 1)
      printf("Stack Overflow");
   else
   stack[top++] = data;
}

int main() {
   push(10);
   push(20);

   printf("%d", stack[top]);
}
```



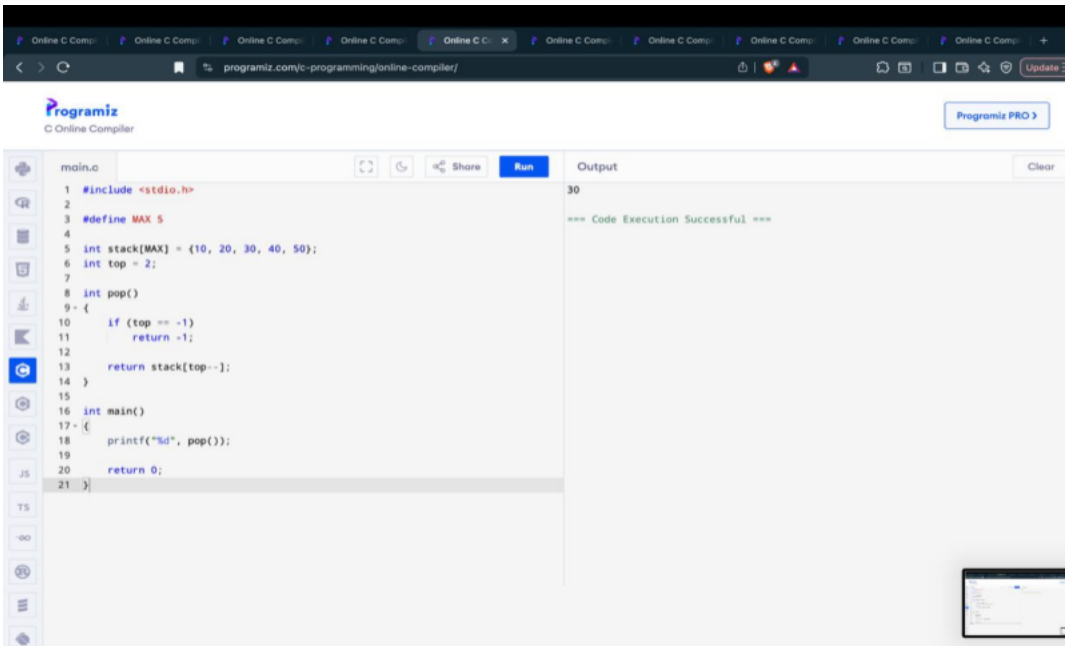
5. **Buggy Code in Stack POP Operation**

```
#include <stdio.h>
#define MAX 5
int stack[MAX];
int top = 2;
int pop() {
   if (top == -1)
      return -1;
   return stack[++top];
}
```



6. **Buggy Code in underflow operation**

```
int pop() {
    if(top == 0) {
    printf("Underflow");
    return -1;
    }
    return stack[top--];
    }
```

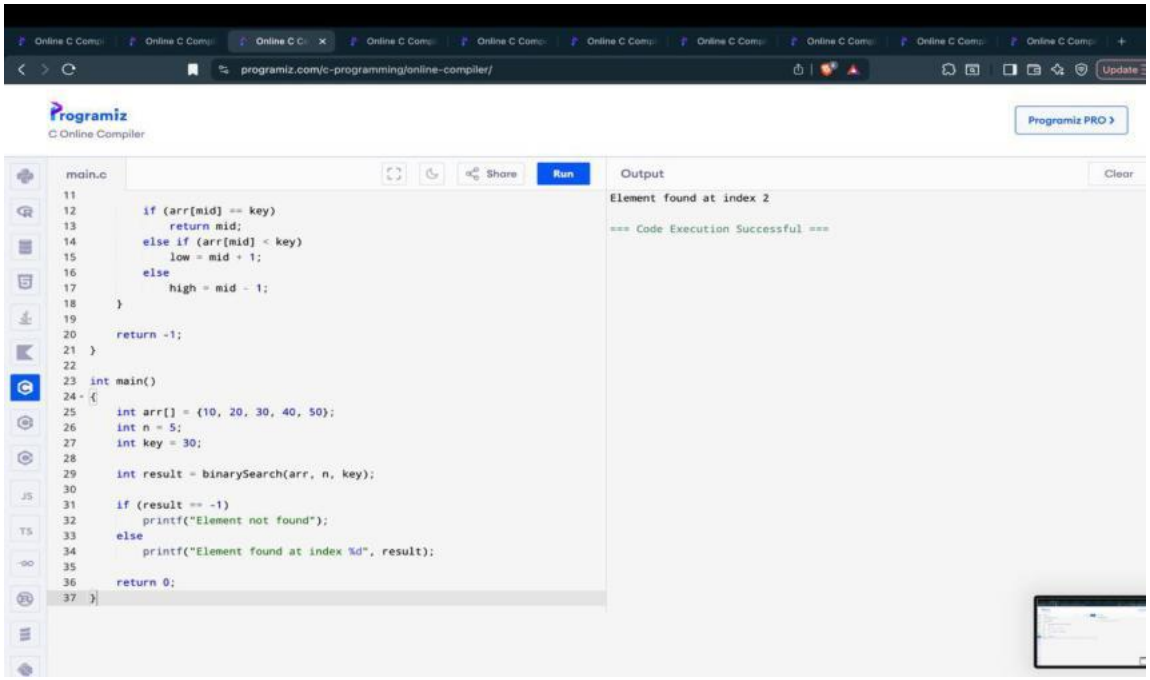


7. **Find the issue in the following code snippet.**

```
#include<stdio.h>

#define MAX 5
void main()
{
int queue[MAX],x;
int front = 0, rear = 0;
printf("Enter the value");
scanf("%d",&x);
void enqueue(int x) {
   if(rear == MAX) {
      printf("Overflow");
      return;
   }
   else{
   queue[rear++] = x;
   printf("Value is %d", queue[rear]);
}}}
```

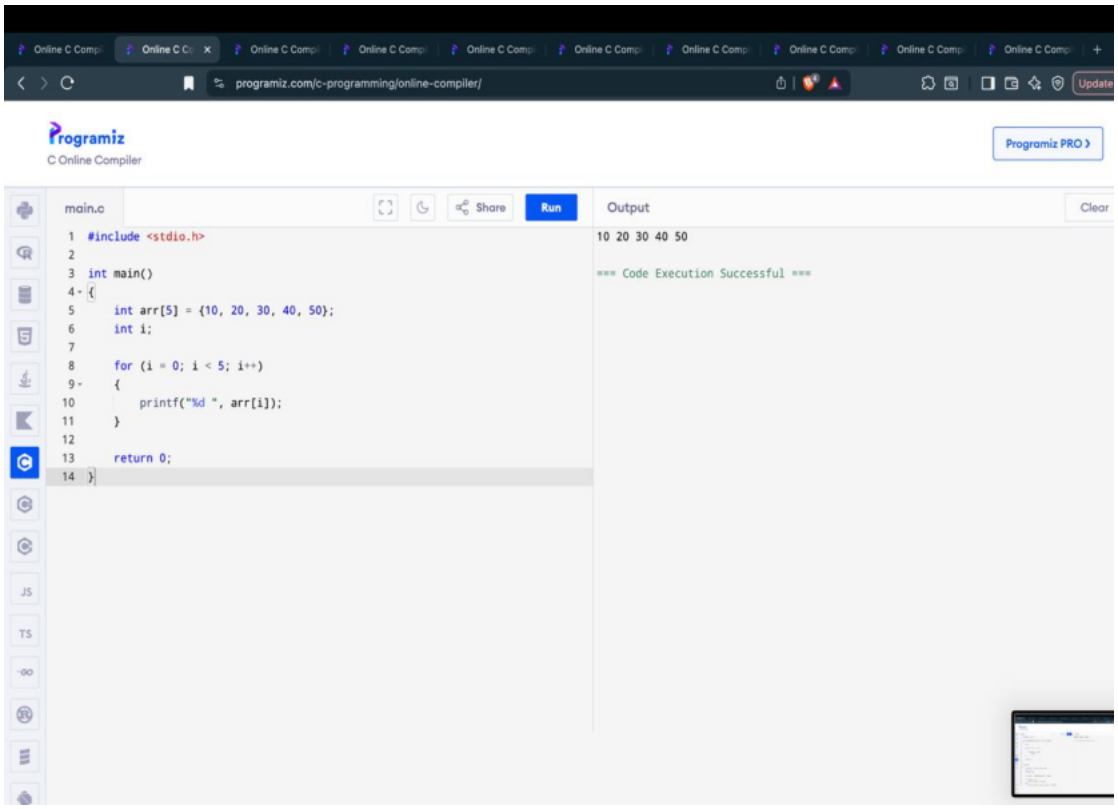
8. **Buggy code in node creation**

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
};

int main() {
   struct Node *newNode;

   newNode->data = 10;
   newNode->next = NULL;

   printf("%d", newNode->data);

   return 0;
}
```



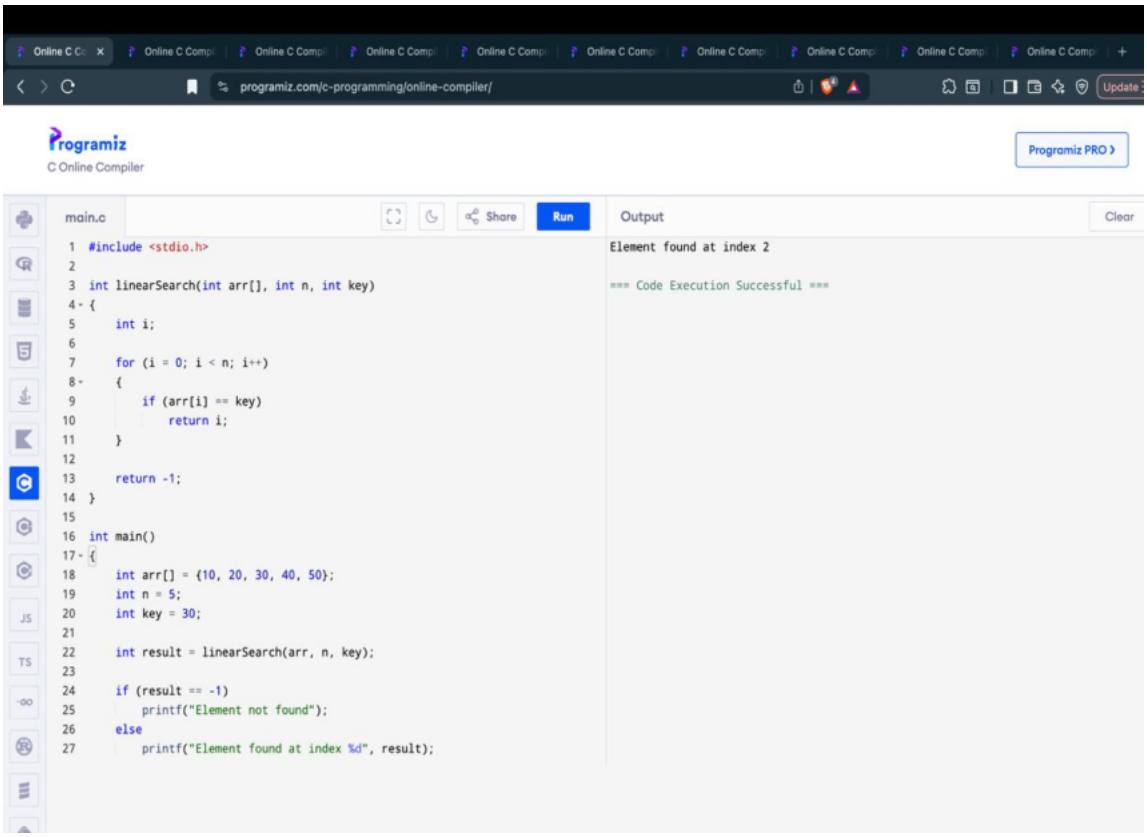
9. **Include missing statements and display the list of elements in linkedlist**

```
void display(struct Node *head)
{
while(head != NULL);
{
printf("%d ", head->data);
head = head->next;
 } }
```



10. Bug in Queue Full Condition-Identify and fix the errors.

```
#define SIZE 5
int queue[SIZE];
int front = -1, rear = -1;

void enqueue(int item)
{
   if(rear == SIZE-1)
   {
      printf("Queue Full");
      return;
   }

   if(front == -1)
      front = 0;

   rear = (rear + 1) % SIZE;
   queue[rear] = item;
}
```



Code of Conduct:

I Shaik Ayesha Tabassum (192525074)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *S.Ayesha*

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation